

37. ChatGPT (LLM Chat Serving Platform)

Interviewer: Design the backend that serves a product like ChatGPT — a user types a message, and a large language model streams back a response, word by word, in a conversational thread. Assume the model itself already exists and is trained; you're designing the **serving infrastructure** that takes a prompt and gets tokens back to the user at scale.

Candidate: Good — I want to be explicit up front that I'm designing *inference serving*, not training. The interesting constraints here are different from a normal CRUD service: each request needs an **expensive GPU**, the model produces output **one token at a time**, and GPUs are the scarcest, most expensive resource in the whole stack. Let me clarify before I design around that.

1. Clarifying the requirements

Candidate: A few questions: 1. Is this single-turn (one prompt, one answer) or multi-turn — does the model need the **prior conversation** as context on every request? 2. Do responses stream token-by-token to the client, or is a single blocking response okay? 3. One model size, or multiple (a fast/cheap model and a slower/smarter one)? 4. Do we need content moderation/safety filtering on both what the user sends and what the model generates? 5. Is GPU capacity effectively fixed (a hard ceiling we must schedule around), or can we assume infinite elastic GPUs?

Interviewer: Multi-turn — every request must include the conversation history as context. Must stream tokens as they're generated, that's core to the UX. Assume multiple model tiers exist. Yes, moderate both directions. And treat GPU capacity as **hard-capped and expensive** — that's the crux of this problem, don't assume you can just add more.

Candidate: Then here's what I'm building:

Functional - `POST /conversations/{id}/messages` → appends the user's message, runs inference with the full conversation as context, **streams** tokens back as they're generated. - Conversation history persists and is replayed as context on every subsequent turn. - Input and output are passed through a **safety/moderation filter**. - Per-user rate limits / quotas (requests per minute, tokens per minute, tiered by plan).

Non-functional - **GPUs are the bottleneck resource** — expensive, capacity-constrained, and each one can serve many users at once *if* we batch cleverly, or almost none if we don't. - **Low time-to-first-token (TTFT)** — the user should see the first word within a few hundred ms, even though the *full* answer may take 5–15 seconds to finish generating. - **Fairness under contention** — a user asking for a 4,000-token essay must not starve ten other users asking a one-line question. - **Graceful overload behavior** — when demand exceeds GPU supply (which will happen), we degrade predictably instead of falling over or letting latency balloon unboundedly. - **Safety** — no unmoderated content reaches the model's output or the user.

2. Estimation — the number that defines the design

Candidate: The number that matters isn't "requests/sec" the way a normal web service counts it — it's **concurrent in-flight generations**, because that's what has to fit in GPU memory and GPU compute at once.

Assume ~20M daily active users chatting; average session sends ~10 messages.
→ 200M messages/day. Spread over active hours (~16h, with peaks), that's a modest ~3,500 requests/sec on average — NOT the hard part.

The hard part: each accepted request doesn't finish in milliseconds like a web request — it occupies a GPU for the ENTIRE generation, which streams over several seconds. So what matters is concurrency, not just arrival rate:

avg generation time ~8s (a few hundred output tokens)
accepted rate ~3,500 req/s (average), bursty to 3–5x at peaks
Little's Law: concurrent in-flight generations = arrival_rate * duration
~3,500 * 8 ≈ 28,000 concurrent generations at once (avg),
peaking well over 100,000 concurrent streams at busy hours.

One GPU node running continuous batching on a mid-size model can typically hold a batch of ~100–250 concurrent sequences (limited by GPU memory for the KV-cache, not by compute). So:

100,000 concurrent generations / ~150 per node ≈ ~700 GPU nodes needed at peak, globally, JUST for decode — and that's the resource we cannot simply autoscale in seconds; GPU fleets are provisioned, not elastic like stateless web pods.

⇒ The whole problem is: GPU decode capacity is fixed and scarce, so the design has to (a) pack as many concurrent sequences onto each GPU as possible (continuous batching), (b) queue/schedule admission so we never accept more concurrent work than GPUs can hold, and (c) stream tokens so perceived latency stays low even though total generation takes seconds.

3. A simple V1 — one request, one GPU, synchronous, no streaming

Candidate: Let me state the naive version so we're aligned on why it doesn't scale.

```
[ Client ] --POST /chat (full prompt)--> [ App server ] --> [ 1 GPU, 1 model instance ]
                                                                (runs forward passes
                                                                token by token, in a
                                                                tight loop, for THIS
                                                                request only)
                                                                |
                                                                <-- full response, once generation finishes --
```

Why it dies at scale: - **One GPU serves exactly one request at a time.** LLM decoding is *autoregressive* — one new token needs one forward pass through the whole model, then the next token needs another, sequentially. At batch size 1, that forward pass is heavily **memory-bandwidth bound**: the GPU spends most of its cycles waiting on weights streaming from HBM rather than doing math, so

utilization is often under 10%. You're paying for a \$30k+ GPU and using a sliver of it. - **No streaming** means the user stares at a blank screen for the full 5–15s generation. - Neither problem is fixable by "add more app servers" — the constraint is the GPU itself.

The rest of this design is about fixing exactly these two things: **pack many concurrent requests onto each GPU (batching)**, and **stream tokens as they're produced**.

4. Scaling up — batching, then a queue in front of GPUs, then multi-region

More users → continuous batching turns idle GPU cycles into throughput

Interviewer: Okay, more users show up. What changes first?

Candidate: The GPU inference servers stop running one sequence at a time and instead run **continuous (a.k.a. dynamic / in-flight) batching**: at every decode step, the server groups the *current* token-generation step of many different, independent conversations into one batched matrix multiply. Frameworks like vLLM / TensorRT-LLM / Triton do exactly this, using **PagedAttention**-style memory management so each sequence's KV-cache (see §7) can live in non-contiguous GPU memory "pages," letting sequences of wildly different lengths share a GPU efficiently without fragmentation.

```
GPU INFERENCE NODE (batched)
step t:  batch = [seqA_tok_i, seqB_tok_j, seqC_tok_k] |
        -> ONE matmul computes the next token      |
            for A, B, and C simultaneously          |
step t+1: seqB finishes -> evicted, seqD admitted |
```

This is the single highest-leverage move in the whole design — it turns a GPU that could serve ~1 user well into one that can serve ~100–250 users well, for roughly the same cost.

Read path — fetching conversation context cheaply

Interviewer: Every request needs the full conversation history as context. How do you serve that without it becoming the bottleneck?

Candidate: This is a small, hot **read** relative to the GPU work: fetch the last N messages for `conversation_id`, ordered by time, format them into the prompt. I store conversation history in a store optimized for exactly this access pattern (see data model, §5) and keep a **short LRU cache** (Redis) of "most recent conversation window" keyed by `conversation_id`, since a user's next message almost always follows their last one within seconds — high cache-hit locality. If the conversation is long enough to exceed the model's context window, I truncate/summarize older turns (a background job can compress old turns into a running summary) rather than trying to stuff everything in.

Write path — persisting messages off the generation critical path

Interviewer: And writing the new message + the model's answer?

Candidate: The user's inbound message is written **before** we call the GPU (so it's never lost even if generation fails). The model's streamed output is buffered server-side and **persisted once as it completes** (with periodic partial-checkpoint writes for very long generations, so a crash mid-stream doesn't lose the whole answer). None of this write blocks the token stream reaching the client — persistence happens alongside/after streaming, not before it, via an async path (queue + workers), the same pattern as the burger/flash-sale docs: never let a durability write gate the user-facing latency.

Multi-region — GPUs are the expensive, geographically lumpy resource

Interviewer: Now go global. What changes?

Candidate: Unlike a normal stateless web tier, **GPU clusters are not cheap to duplicate everywhere** — they're provisioned in specific data centers with GPU supply. So: - **Stateless edge (API gateway, auth, moderation input filter, rate limiter)** is deployed in every region — cheap, and it's on the latency-critical path for TTFT. - **GPU inference clusters** live in a smaller number of regions with GPU capacity; requests are routed to the *nearest region with available capacity*, not necessarily the nearest region geographically — if the closest region's GPUs are saturated, we spill to the next closest rather than queue locally and blow latency. - **Conversation store** is regional-primary with async cross-region replication for durability/failover, since a user's own conversation is almost always read/written from one region at a time (session affinity) — we don't need multi-region strong consistency on chat history, just durability.

5. API + data model

API - `POST /conversations/{id}/messages` — body: `{content}`. Response: **SSE stream** of `{token}` events, ending with a `{done, message_id, usage}` event. - `GET /conversations/{id}/messages?before=...` — paginated history for rendering the thread. - `DELETE /conversations/{id}` — soft-delete.

Data model — two very different stores for two very different access patterns

Conversation/message store — access pattern is: **append a message to a conversation, and read back the last N messages of a conversation, ordered**. No joins, no multi-row transactions, but *enormous* write volume (hundreds of millions of messages/day) and it must scale horizontally without a single-writer bottleneck.

```
conversations(conversation_id PK, user_id, created_at, title, ...)
messages(conversation_id PARTITION KEY, seq_no SORT KEY, role, content,
         token_count, model, created_at)
```

I pick a **wide-column / key-partitioned NoSQL store (DynamoDB or Cassandra)**, partitioned by `conversation_id`, sorted by `seq_no`. Why, and why not Postgres: a single Postgres primary can absolutely handle this at moderate scale, but at hundreds of millions of appends/day across millions of independent conversations, the *access pattern itself* — partition-keyed appends and ordered range reads, no cross-conversation joins — is exactly what a wide-column store is built for, and it scales writes horizontally by adding nodes rather than by manually sharding a relational cluster.

Account / billing / quota store — this **does** need ACID: plan tier, quota counters that must not double-charge, payment records. I use **PostgreSQL** here — low volume (one row per user, not per message), and correctness matters far more than raw write throughput.

Why not one database for both? Because they have opposite shapes: messages are high-volume, append-only, no-transaction-needed data; billing is low-volume, transactional, correctness-critical data. Forcing both into Postgres makes the message table dominate and risks contention on a system that needs ACID guarantees for money; forcing both into DynamoDB makes billing correctness (transactions, strong consistency) awkward. Use the right tool for each shape.

6. Latency, consistency, async, caching

Latency — the metric that matters is **time-to-first-token (TTFT)**, not total completion time. TTFT is dominated by: queueing for a GPU slot + prompt "prefill" (processing the input prompt once, which is compute-bound and fast) before decode (token-by-token) begins. We target TTFT in the low hundreds of ms; total time is however long the answer is, but the user perceives it as fast because tokens are already streaming.

Consistency - Strong / read-your-writes: a user's own conversation — they must see their own message and the model's reply immediately after sending. Session-affinity to one region gives this for free. - **Eventual:** cross-device sync of conversation history, usage/quota dashboards, analytics. A few seconds of lag on "how many tokens have I used this month" is fine.

Async off the critical path: durable persistence of the completed message, updating billing/usage counters, analytics events, and any post-hoc safety re-scan of the full transcript all happen via a queue (Kafka) after the stream completes — none of them delay a token reaching the user.

Caching — two very different caches, for two very different things

1. **KV-cache (per-request, in GPU memory)** — not really a "cache" in the CDN sense, but the core efficiency primitive: as the model generates token n , it needs the attention key/value tensors from tokens $1..n-1$. Recomputing those from scratch every step would be quadratically wasteful; instead they're cached in GPU memory and reused, only the new token's KV is computed each step. **Prefix caching** takes this further: if many requests share an identical prefix (e.g., the same system prompt, or a long shared document), the KV-cache for that shared prefix is computed once and reused across requests — this is what products mean by "prompt caching," and it can cut cost/latency dramatically for repeated-prefix workloads (agents, few-shot prompts, long system instructions).
2. **Semantic response cache (cross-request, e.g. Redis + embeddings)** — for near-duplicate prompts (common FAQs, popular queries), cache the embedding of the prompt and, on a new request, check for a close-enough vector match; if found, return the cached response and skip the

GPU entirely. This is a pure cost/latency win but must be used carefully — only for content where staleness/exactness risk is acceptable (never for anything personalized, time-sensitive, or safety-adjacent), with a TTL and explicit invalidation.

7. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
GPU inference serving + batching	vLLM/TensorRT-LLM/Triton-style server with continuous (in-flight) batching + PagedAttention	Packs many concurrent decode steps into one batched matmul, turning a memory-bandwidth-bound, ~10%-utilized GPU into one running near its compute ceiling. <i>Not one-request-per-GPU</i> — see Deep-dive A, it wastes ~90% of the most expensive resource in the stack.
Admission queue in front of GPUs	Priority queue + admission control (token-budget aware)	GPU batch slots are finite; a queue absorbs bursts and lets us reject/degrade before wasting a GPU slot on a request we can't finish on time. <i>Not "just accept everything and let the batcher sort it out"</i> — that lets queueing delay spiral unbounded under overload (see Deep-dive B).
Conversation/message store	DynamoDB / Cassandra (wide-column, partitioned by conversation_id)	Matches the access pattern exactly (partitioned append + ordered range read); scales writes horizontally. <i>Not a single Postgres primary</i> — hundreds of millions of appends/day across independent conversations don't need cross-row transactions, and forcing them through one relational writer just to get SQL is a self-inflicted bottleneck.
Account/billing/quota store	PostgreSQL	Low volume, correctness-critical (no double-charging, accurate quota); ACID transactions matter far more than raw throughput here.
KV-cache / prefix cache	In-GPU-memory KV-cache with PagedAttention; shared-prefix caching for repeated system prompts	Avoids recomputing attention over the whole prefix every token; sharing across requests with identical prefixes multiplies the win. This is a serving-layer primitive, not optional — without it, decode cost per token grows with context length on every single step.
Semantic response cache	Redis + embedding similarity lookup	Skips the GPU entirely for near-duplicate prompts — pure cost savings. <i>Not caching everything</i> — anything personalized, time-sensitive, or safety-relevant must always hit the model fresh.
Streaming transport	Server-Sent Events (SSE) over HTTP (WebSocket as an alternative for bidirectional needs)	One-directional server→client token stream is exactly SSE's use case: simpler than WebSocket, works over plain HTTP/2, auto-reconnect semantics, easy to proxy through standard load balancers/CDNs. <i>Not a single blocking response</i> — see Deep-dive A on perceived latency.

Concern	Choice	Why this, and why not the alternative
Event/async backbone	Kafka	Durable, replayable stream for "persist completed message," "update usage counters," "post-hoc safety audit" — none of which can be allowed to slow down the live token stream.
Rate limiting / quotas	Redis token-bucket per user/API key, tiered by plan	O(1) atomic ops at memory speed, same pattern as a dedicated rate-limiter service; naturally shards by user key so no hot spot.
Moderation filter	Lightweight classifier model in the request path (input) + streaming classifier on output chunks	Must run before the expensive model call (reject bad input cheaply) and continuously during streaming (abort generation if the output veers unsafe) rather than only after the fact.

The one-sentence justification you'd say out loud: *"Continuous batching is what makes each GPU economically viable — it's the difference between serving one user and a few hundred per node; a priority admission queue in front of that batching is what keeps GPU capacity from being overrun during spikes; streaming turns a multi-second generation into a fast-feeling experience by optimizing for time-to-first-token instead of total completion time; and the conversation store is a wide-column NoSQL system because the access pattern — append and ordered-read per conversation — is exactly what that shape of database is built for, while billing stays on Postgres because that data needs ACID, not throughput."*

8. Failure modes & graceful degradation

Failure	What happens	Mitigation
GPU node crashes mid-generation	In-flight streams on that node die	Client SDK detects stream drop, retries against a healthy node with the same context; partial output already streamed is preserved client-side; server marks the message truncated if not resumed
GPU fleet saturated (demand > capacity)	New requests would queue indefinitely	Admission control rejects/queues with an honest "high demand" signal past a bounded wait; optional fallback to a smaller/cheaper model tier (see Deep-dive B)
Conversation store (DynamoDB/Cassandra) partition slow/down	Can't fetch history for affected conversations	Serve from the Redis recent-context cache if present; degrade to "no history" only as last resort with a clear notice, never silently drop context
Moderation service down	Can't safety-check input/output	Fail closed for high-risk categories (reject/queue), fail open only for low-risk, well-audited paths — never silently skip moderation
Kafka lag	Persistence/usage-counter updates delayed	User unaffected (already has their tokens); billing counters catch up asynchronously; cap on how stale quota enforcement is allowed to get
Regional GPU cluster overloaded	Requests routed there would queue	Cross-region spillover to a region with spare GPU capacity, at the cost of slightly higher latency

9. Follow-up curveballs

Interviewer: A user asks the same question 10,000 other users asked today verbatim (e.g., "what's the capital of France"). Do you really run the model 10,000 times?

Candidate: For a purely factual, non-personalized, non-context-dependent prompt, the semantic cache (§6) can serve a cached answer without touching a GPU at all. The key discipline is scoping it tightly: only cache when there's no conversation context, no personalization, and low risk if the cached answer is slightly stale — and always cache the model's own output, never something that bypasses moderation.

Interviewer: How do you support a "regenerate this response" button?

Candidate: Treat it as a new generation request with the same prompt context but excluding the previous (unwanted) assistant turn, tagged with a `regenerate_of: message_id` link so both versions are retained in history (useful for feedback/training signal later). It goes through the exact same queue → batch → stream path as any other message.

Interviewer: Some users are on a paid tier and expect priority. How does that show up in the design?

Candidate: The admission queue (§8/Deep-dive B) isn't FCFS-only — it's a **priority queue** keyed by plan tier, so paid requests jump ahead of free-tier requests when GPU slots are contended, while still respecting a max-wait bound for free tier so it never starves completely. Rate limits (§6) are also tier-specific token buckets.

Interviewer: How would you A/B test a new model version safely?

Candidate: Route a small percentage of traffic (by user bucket, sticky per user) to the new model's GPU pool, keep it capacity-isolated from the main fleet so a bad rollout can't starve production GPUs, and compare quality/latency/safety metrics before ramping the percentage up. Same blast-radius-isolation instinct as the flash-sale doc's separate cluster.

10. Deep-dive: "Why not one-request-per-GPU, and why not return the whole answer at once?"

Interviewer: Both of these feel like they'd simplify the system a lot. Convince me they're wrong.

Candidate: They fail for two different reasons — one is a GPU-economics problem, one is a perceived-latency problem — but they compound into the same bad outcome: an expensive, slow-feeling product.

Why not one-request-per-GPU (no batching)

Autoregressive decoding computes one token per forward pass. At **batch size 1**, that forward pass has to stream the *entire* model's weights through the GPU for a tiny amount of actual math (one token's worth) — it's **memory-bandwidth bound**, not compute bound, so the GPU's massive FLOPs sit mostly idle waiting on memory. Continuous batching groups many different requests' *simultaneous* decode steps into one shared forward pass: the memory transfer (weights) is amortized across all of them, while the compute scales with batch size, so you get vastly more useful compute per byte of memory bandwidth spent — which is exactly the resource that was the bottleneck.

Concern	One-request-per-GPU (no batching)	Continuous batching
GPU utilization during decode	Often <10% — memory-bandwidth bound, compute mostly idle	Much higher — shared forward pass amortizes memory traffic across many sequences
Concurrent users per GPU	~1	~100–250 (model/hardware dependent)
Cost per generated token	High — you're paying full GPU cost for ~10% utilization	Much lower — same GPU, far more useful work per dollar
Effect of a long response	Blocks that GPU entirely until done	Shares the GPU with other sequences step-by-step; doesn't block the <i>node</i> , though see Deep-dive B for the per-slot fairness nuance
Complexity	Trivial to implement	Needs a batching scheduler + paged KV-cache memory manager

Why not return the whole response at once (no streaming)

Generation is *inherently* sequential — token 50 can't exist before token 49 is computed — so a 500-token answer takes however long 500 sequential decode steps take, often 5–15 seconds, **no matter how the transport works**. The transport choice only affects *when the user sees it*:

Concern	Full response at once	Token streaming (SSE)
What the user sees during generation	Nothing — blank/spinner for the full duration	Words appearing as generated
Perceived latency metric	Total completion time (5–15s)	Time-to-first-token (~hundreds of ms)
Early cancel	Impossible — must wait for the full generation or throw it away	User can stop reading/cancel, freeing the GPU slot immediately
Network/memory	One large payload at the end	Small chunks over time; slightly more transport overhead, negligible at this scale
Failure mid-generation	All-or-nothing — a crash loses everything	Partial output already delivered and can be preserved

Streaming doesn't make the model *faster* — it makes the **wait feel productive**, and it gives you a cancel point that saves GPU work when users don't need to see the rest.

Rule of thumb: Batch when the bottleneck is a scarce compute resource being underutilized by small individual units of work; stream when the bottleneck is an inherently sequential process and the fix is exposing partial progress rather than making the process itself faster. GPU serving needs both, for two unrelated reasons.

11. Deep-dive: scheduling scarce GPUs — head-of-line blocking, disconnects, and overload

Interviewer: Walk me through the hardest part in detail: GPU capacity is fixed, demand spikes, and requests have wildly different lengths. How do you schedule fairly, avoid one giant generation blocking small ones, handle a user who closes their laptop mid-stream, and keep the system alive when demand exceeds supply?

Candidate: This is really three coupled problems: **batch composition** (who shares a GPU step-by-step), **admission** (who's even allowed to start), and **cleanup** (reclaiming resources when a request stops mattering).

The naive batching mistake: static batches cause head-of-line blocking

If you batch the *old* way — collect N requests, run all of them to completion together as one fixed batch — then one user asking for a 4,000-token essay forces everyone else in that batch to wait for all 4,000 steps, even if their own answer finished at step 30:

```
STATIC BATCH (bad):
step:          1    2    3    ... 30          ...          4000
seqA(long) : t1   t2   t3          t30 ..... t4000 (finally done)
seqB(short): t1   t2   t3          DONE at 30, but batch can't
                                     return control until A finishes
                                     -> B's tokens are held up / GPU slot wasted on B
```

The fix: continuous (iteration-level) batching

The scheduler re-decides the batch **every single step**, not once per whole request. The moment seqB emits its end-of-sequence token, it's evicted and its slot is immediately handed to the next queued request:

```
CONTINUOUS BATCH (good):
step:          1    2    3    4    5    ...
slot 1:      seqA  seqA  seqA  seqA  seqA  (long generation, runs alongside others)
slot 2:      seqB  seqB  DONE  seqD  seqD  (B finishes at step 3, D admitted at step 3)
slot 3:      seqC  seqC  seqC  DONE  seqE  (C finishes at step 4, E admitted at step 4)
```

Now a long generation shares GPU cycles with short ones step-by-step, instead of blocking them outright — this is what actually solves head-of-line blocking, not a bigger batch size.

Admission — a queue in front of the batch, sized to the batch's real capacity

Continuous batching still has a **finite number of concurrent slots** (bounded by GPU memory for KV-caches). A queue sits in front, admitting new requests only as slots free up:

```
[ Incoming requests ]
      |
[ Priority admission queue ] -- ranked by: paid tier > free tier,
      |                               then by estimated cost (short prompts admitted
      |                               preferentially when near saturation)
      v
[ GPU node: continuous-batch scheduler, N concurrent slots ]
      | slot frees on completion / cancel → admit next queued request
      v
[ streaming tokens back to clients ]
```

Without this queue, an overloaded batcher would try to cram in every incoming request, degrading everyone's per-step latency simultaneously instead of cleanly queueing the excess.

Handling a user who disconnects mid-stream

If the client closes the connection (tab closed, network drop) partway through generation, the GPU keeps computing tokens nobody will ever see unless we detect it:

```
t=0s  user sends prompt, stream starts
t=2s  user's laptop closes / connection drops
t=2s.. GPU slot keeps generating into a socket write that now fails/buffers
-> WITHOUT detection: the slot is wasted for the rest of that generation,
    denying it to a real waiting request.
```

Fix: the streaming layer detects a closed connection (failed write / TCP close / missing heartbeat) within a short bound and immediately **cancels the generation and reclaims the slot** — evict the sequence from the batch, free its KV-cache pages, hand the slot to the next queued request. We also persist whatever was generated so far (marked truncated) so the conversation history isn't silently incomplete, and don't bill/count tokens generated purely after disconnect if the product's usage policy says so.

Ranked fixes for scheduling under scarcity, worst-to-best foundation

1. **Continuous/iteration-level batching** — the non-negotiable base fix for head-of-line blocking; without it nothing else here matters much.
2. **Priority queues + admission control** — protects the batch from being overrun and lets paid/urgent traffic be served preferentially without starving free tier outright.
3. **Fast disconnect detection + slot reclamation** — stops wasted GPU work from silently eating capacity that queued, real requests need.
4. **Smaller/faster fallback model under sustained overload** — when even a well-scheduled fleet is saturated, degrade some traffic to a cheaper, smaller model (faster per-token, lower quality) rather than let queue wait times balloon for everyone; make this an explicit, monitored decision, not a silent quality regression.
5. **Cross-region/cross-cluster overflow and autoscaling** — the slowest lever (provisioning GPUs takes real time), so it's the backstop for sustained demand growth, not the spike response.

Recommended approach

Lead with **continuous batching (#1)** as the architectural foundation — it's what makes the whole GPU fleet efficient and fair at the same time. Layer **priority admission (#2)** on top so scarcity is allocated by policy, not chaos. Make **disconnect reclamation (#3)** a hard requirement — an abandoned generation stealing a slot from a real waiting user is pure waste. Reserve **model-tier fallback (#4)** as the explicit overload valve, and treat **scaling out (#5)** as the long-term capacity fix, not something you can lean on during a spike.

What to say out loud: "Continuous batching is what turns 'one long request blocks everyone' into 'the scheduler re-decides the batch every step,' which is the real fix for head-of-line blocking — not just a bigger batch. On top of that, a priority queue decides who gets a slot when slots are scarce, fast disconnect detection makes sure we never burn GPU cycles generating for nobody, and a smaller fallback model is the deliberate degrade valve when even that isn't enough — because GPUs can't be autoscaled in seconds, the only real levers during a spike are scheduling and admission, not more hardware."

Summary — the mental model

Serving an LLM chat product is **not a normal web-scaling problem — it's "pack as much concurrent, GPU-hungry, inherently sequential work as possible onto a fixed, expensive compute resource, without starving anyone or wasting a slot on someone who's already gone."** The winning moves: **continuous batching** turns idle, memory-bandwidth-bound GPUs into efficient shared compute; a **priority admission queue** decides who gets a scarce slot instead of letting the batcher get overrun; **token streaming** makes a multi-second sequential process feel fast by optimizing time-to-first-token instead of total completion time; the **KV-cache and prefix caching** avoid redoing attention work that's already been done; a **wide-column conversation store** matches the true access pattern of chat history; and **fast disconnect detection plus a smaller-model fallback** are the graceful-degradation valves for when even good scheduling can't keep up with demand. Everything bends toward one fact: the GPU is the scarce, expensive resource, and the entire serving stack exists to keep it maximally, fairly, and safely utilized.